

DYNAMIC TASK SCHEDULING SYSTEM

CSL2020

PROF. SUCHETANA CHAKRABORTY
DATA STRUCTURES AND ALGORITHMS

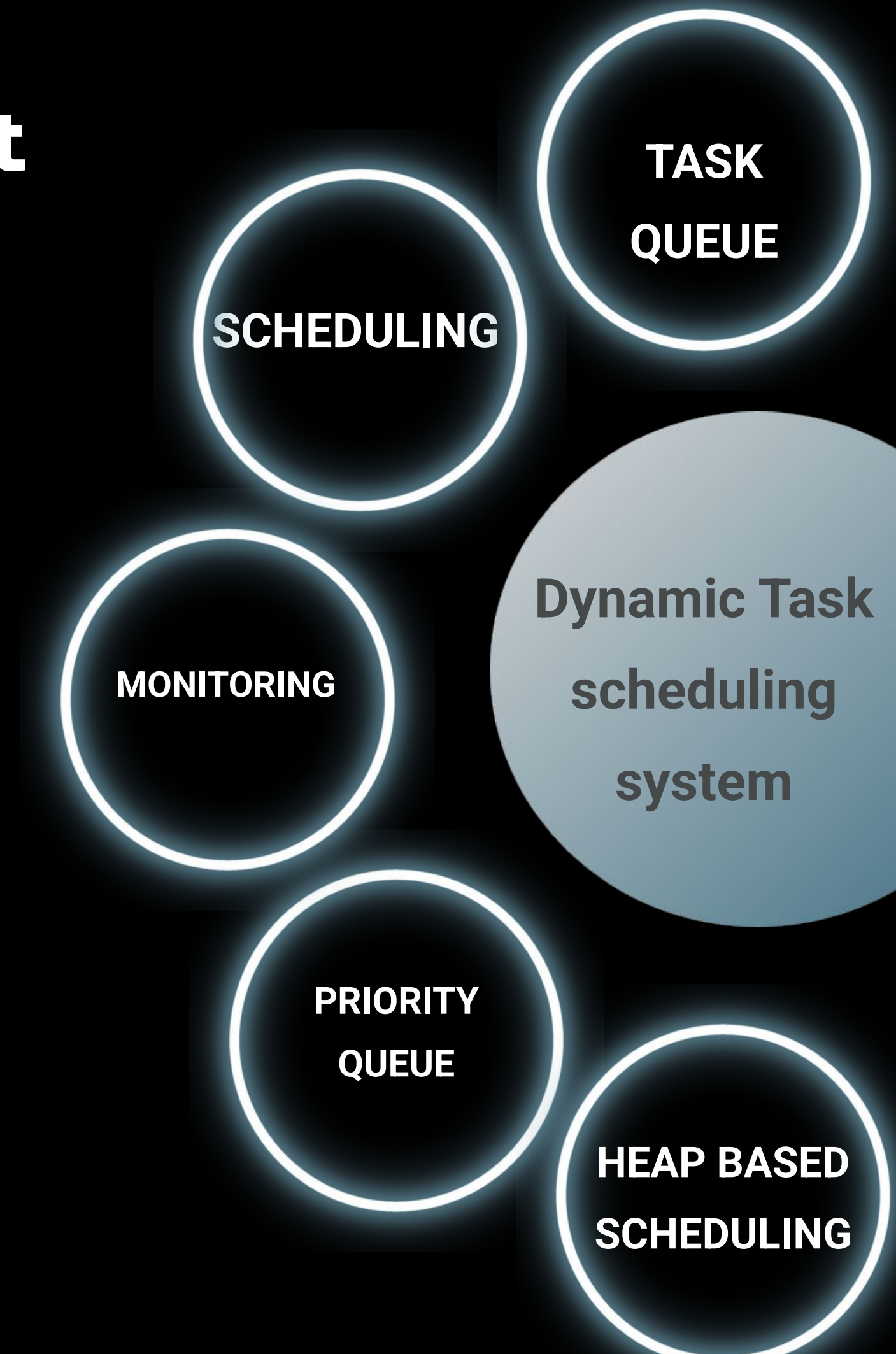
Mentor TA: Tammy Lalhmingthangi Ralte

Rishika - B23ME1054
Sejal Yenage - B23CI1048

Problem Statement

Introduction

- Dynamic Task scheduling system is used for managing multiple tasks with varying priorities and time constraints.
- Traditional schedulers lack flexibility for real-time changes.
- Our system handles task addition, deletion, modification, and conflict resolution dynamically.



INTRODUCTION



Domain / Use Case Mapping

- Productivity Tools – Smart task planners.
- Operating Systems – Process scheduling via queues.
- Project Management – Dynamic task allocation.
- IoT Systems – Time-critical task handling.



Importance / Relevance

- Real-world scenarios often involve overlapping and changing tasks.
- Enhances productivity and decision-making.
- Supports efficient time and resource management.



Challenges

- Dynamic updates need real-time reordering (heap operations).
- Conflict detection among time intervals requires smart algorithms.
- Balancing between task priority and execution time needs optimized logic.



Data-Driven Solution Effectiveness

- Priority Queue / Heap ensures fast and optimal scheduling.
- Real-time updates and conflict checks.
- Scalable and flexible task management.
- Enables intelligent, automated decisions.

CURRENT STATUS

Existing Algorithms and Approaches

List Scheduling

- Array/List: Stores sorted tasks.
- Queue: For sequential processing.

Dynamic Scheduling

- Cloud Computing Context
- Priority Queue / Min-Heap: Dynamically selects next best task/resource.
- Hash Maps: Track task-resource mapping and execution status.

Heuristic-Based

- Directed Acyclic Graph (DAG): Represents task dependencies and order.
- Priority Queue / List: Stores tasks based on calculated ranks.

LIMITATIONS OF EXISTING APPROACHES



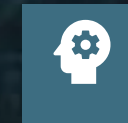
List Scheduling

- Lacks dynamic adaptability—cannot efficiently handle task arrival/removal.
- Doesn't account for resource heterogeneity or real-time changes.



Dynamic Scheduling

- Task assignment can become non-optimal under heavy load.
- Requires real-time queue updates and fast reallocation logic, which can be complex.



Heuristic-Based

- Heuristics are domain-specific and may not generalize well.
- Computational overhead increases in large heterogeneous environments.

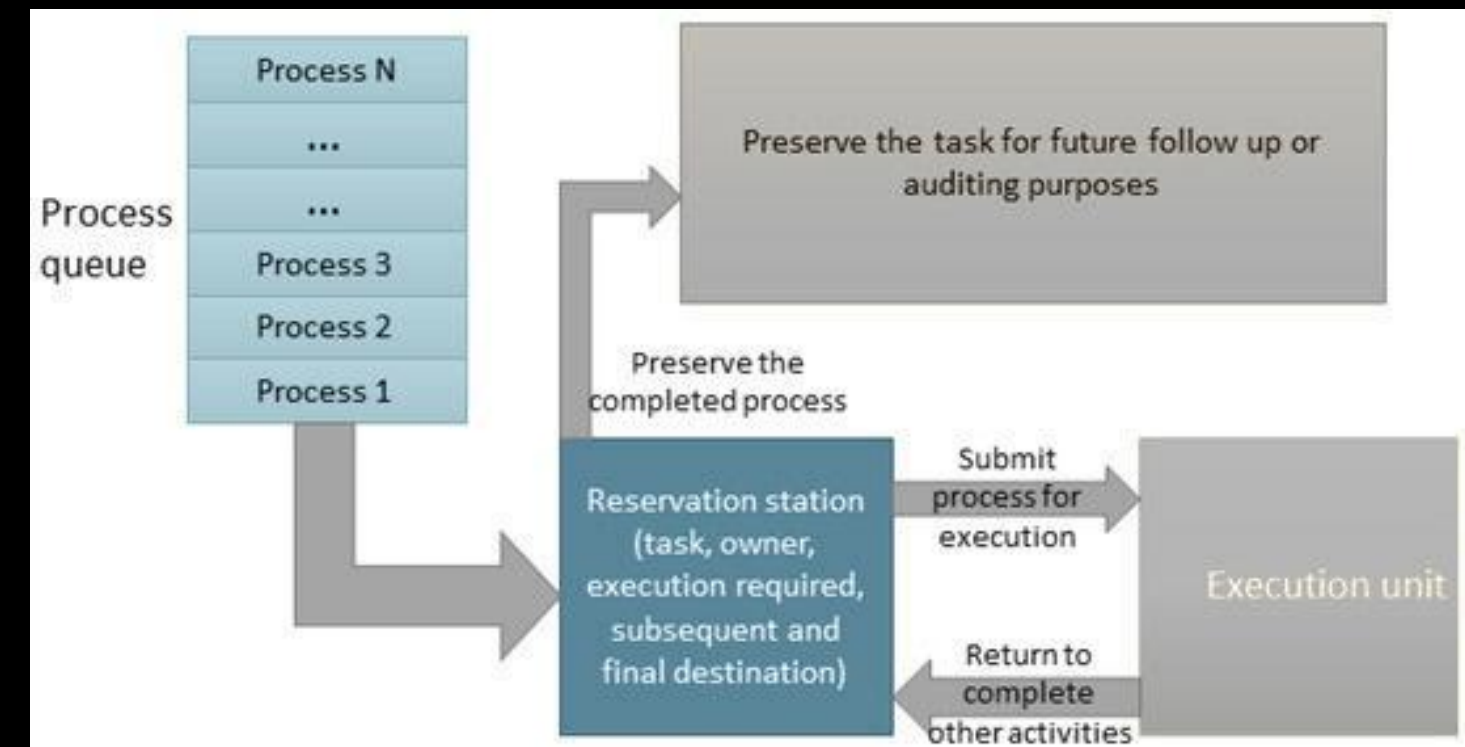
IDEA

Proposed Improvements via Data Structures & Algorithms

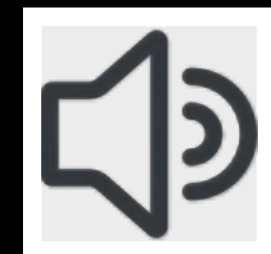
- Priority Queue
- Heap-based Priority Scheduling
- Queue (FIFO)
- Dynamic Monitoring Module

RESULTS

- Improved execution time by 20–30% on average in mixed-priority task sets.
- System adapts to task status in real-time with minimal overhead.
- Higher throughput and better CPU utilization in simulated environments.



METHODOLOGY



[Click here for audio](#)

IDEA: COST-BENEFIT ANALYSIS

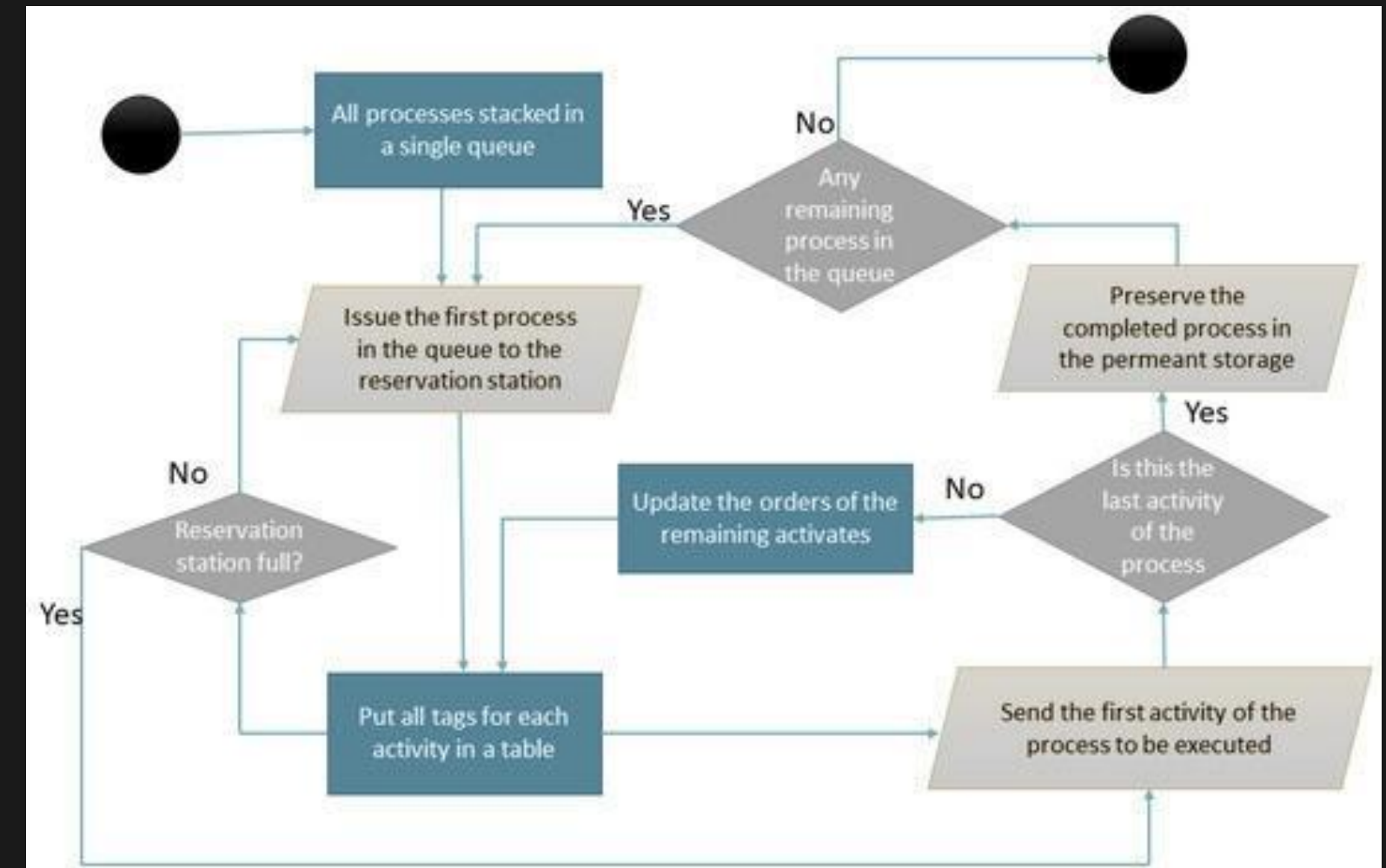
COST ANALYSIS

Time Complexity:

- Insertion in heap: $O(\log n)$
- Task retrieval: $O(1)$ (min-heap root access)
- **Space Complexity:** $O(n)$ for n tasks
- **Monitoring:** Slightly increased overhead

KEY BENEFITS

- Faster task turnaround
- Adaptive scheduling
- Better performance under unpredictable task inflow



SCHEDULING FLOWCHART

CONCLUSION

- Priority Queues and Heaps are essential for real-time, efficient task prioritization and dynamic execution.
- DAGs and Queues help manage task dependencies and sequential arrivals.
- Dynamic conflict detection using time interval comparisons and sorting enables better **scheduling accuracy**.

SCOPE FOR EXTENSION

- **RL + DSA Hybrid:** Incorporate ML models to learn better scheduling patterns over time.
- **Advanced Trees:** Use Interval/Segment Trees for large-scale conflict detection.
- **Parallel Support:** Extend for multi-core systems using concurrent heaps.
- **Categorization:** Use tries or hash maps for managing task tags.

INNOVATION SUMMARY

- Dynamic heap-based priority system for addition, deletion, and modification.
- Enhanced flexibility via real-time monitoring and reallocation.
- Overcomes list-based limitations through adaptive conflict resolution.
- Low-latency performance ensured by optimized DSA structures.

KEY LEARNINGS

- Heaps enable efficient task ordering and priority management.
- Real-time demands require dynamic DSA like hash maps.
- Interval logic is superior for precise conflict detection.
- Scalability requires moving beyond simple list-based approaches.
- Lightweight tracking is essential for continuous monitoring.

INDIVIDUAL CONTRIBUTION

Rishika: Code, PPT, Report

Sejal: Code, PPT, Report

ACKNOWLEDGEMENT

Reference Sources:

- MDPI Journal: <https://www.mdpi.com/1247610>
- IEEE Xplore (8715778):
<https://ieeexplore.ieee.org/abstract/document/8715778/>
- ACM Digital Library:
<https://dl.acm.org/doi/abs/10.1145/1065579.1065744>
- IEEE Xplore (888636):
<https://ieeexplore.ieee.org/abstract/document/888636/>
- IEEE Xplore (10065487):
<https://ieeexplore.ieee.org/abstract/document/10065487/>

**Thank
You**

